Accelerating Post-Quantum KEM HQC on ARMv9-A using SVE2 and NEON

***Abstract*— abstract abstract**

***Index Terms*— 111.**

I. Introduction

The emergence of quantum computers poses a fundamental threat to classical public-key cryptosystems such as RSA and Elliptic Curve Cryptography (ECC), whose security relies on the hardness of integer factorization and discrete logarithm problems, both of which can be solved in polynomial time by Shor's algorithm [1]. In response, the National Institute of Standards and Technology (NIST) launched a post-quantum cryptography (PQC) standardization initiative in 2016 to identify quantum-resistant algorithms. After multiple rounds of evaluation, the first PQC standards were published in 2024, including ML-KEM [2], ML-DSA [3], SLH-DSA [4], and FN-DSA (Falcon) [5]. In the fourth round of the standardization process [6], NIST evaluated three code-based Key Encapsulation Mechanism (KEM) candidates – Classic McEliece [7], Bit Flipping Key Encapsulation (BIKE) [8] and Hamming Quasi-Cyclic (HQC) [9]. Classic McEliece offers conservative security but suffers from large public key sizes. BIKE provides smaller keys but has an insufficiently analyzed decryption failure rate. Ultimately, in 2025, NIST selected HQC as an additional KEM standard, owing to its mature security analysis and the need for cryptographic diversity. Unlike lattice-based schemes such as ML-KEM, HQC is a code-based KEM whose security relies on the hardness of the syndrome decoding problem for quasi-cyclic codes, a problem believed to be hard even for quantum computers. Since HQC operates over the binary field, optimization techniques developed for prime-field schemes cannot be directly applied to it. Binary polynomial multiplication dominates the overall execution time across all three KEM procedures, making its efficient implementation on platforms such as embedded microcontrollers and modern ARM processors a critical research challenge.

ARM processors power a wide range of devices, from low-power embedded systems and IoT devices to high-performance mobile and edge computing platforms, making them a critical target for practical PQC deployment. As computational demands continue to grow, the ARM architecture has continuously evolved to deliver higher performance. The ARMv9-A architecture [10] introduces the Scalable Vector Extension 2 (SVE2) [11] as a key advancement over previous generations. Unlike the fixed 128-bit width of the Advanced SIMD extension (NEON), SVE2 supports variable-length vector registers ranging from 128 to 2048 bits, enabling more flexible and efficient data-level parallelism. These features make ARMv9-A an attractive platform for accelerating PQC algorithms. Recent works have begun using SVE2 to optimize ML-KEM and ML-DSA [12], [13] on the Apple M4 Pro SoC, the first publicly available consumer-grade processor to support SVE2. However, HQC has not yet been explored on the ARMv9-A architecture. This work addresses this gap by presenting the first HQC implementation on this platform, leveraging both NEON and SVE2 vectorization. It demonstrates that code-based PQC can be efficiently deployed on ARMv9-A processors, providing a foundation for future optimizations and practical adoption of HQC on ARM platforms.

A. Related Work

HQC has been implemented and optimized across a range of platforms, with polynomial multiplication being the main focus of optimization efforts. On x86 architectures, Robert and Véron [14] systematically explored Toom-Cook and Karatsuba decompositions combined with the `VPCLMULQDQ` instruction under AVX-512, reducing polynomial multiplication cycles for HQC and BIKE by up to 39%. Dong et al. [15] proposed OptHQC, achieving an average 55% speedup over the reference implementation through sparsity exploitation, AVX2-accelerated hashing and lookup-table-based multiplication. Chen et al. [16] further accelerated HQC by combining Frobenius Additive Fast Fourier Transform (FAFFT) with Karatsuba, and demonstrated that Galois Field New Instructions (GFNI) support yields significant speedup over AVX2+`PCLMULQDQ` implementations.

ARM processors are widely deployed in mobile and embedded devices, making them a critical platform for practical PQC deployment. On ARM Cortex-M4, Chen et al. [17] first applied FAFFT to optimize polynomial multiplication in BIKE, establishing the foundational approach for code-based PQC on embedded ARM. Building on this, Lee et al. [18] presented the first HQC implementation on Cortex-M4 by reformulating the fixed-operand multiplications in FAFFT as binary matrix-vector products, achieving 80%–92% speedup in polynomial multiplication and up to 84% overall KEM improvement over PQClean. Kim et al. [19] explored optimal Toom-Cook and Karatsuba combinations for polynomial multiplication on Cortex-M4. Jang et al. [20] addressed the padding overhead of non-power-of-two polynomial lengths via Hybrid FAFFT-Chinese Remainder Theorem (CRT) and Hybrid FAFFT-Karatsuba methods, reducing polynomial multiplication cycles by 33.4% and 27.2% for HQC-1 and HQC-3, respectively. On ARM A-profile processors, Chen et al.

[16] proposed combining FAFFT with Karatsuba on Apple M1 and a Kronecker Segmentation with CRT approach on ARM Cortex-A72, alongside a key caching strategy to reduce redundant computations in encapsulation and decapsulation.

Recently, several PQC schemes have been optimized for the ARMv9-A architecture. Wei et al. [12], [13] optimized ML-KEM and ML-DSA on the Apple M4 Pro. For ML-KEM, they proposed two NTT variants, VecNTT and MatNTT, using SVE2 and SME respectively, achieving up to 7.77× NTT speedup. For ML-DSA, they proposed a parallel sparse polynomial multiplication (PSPM) scheme combined with a predicate-based early rejection mechanism, achieving a 70–72% improvement in signing performance. While HQC has also been implemented on ARM platforms, existing work relies on fixed-width NEON instructions, and no prior work has investigated HQC on the ARMv9-A architecture, whose SVE2 extension introduces scalable vector registers and enables more flexible data-level parallelism. The ARMv9-A architecture is increasingly adopted in commercial processors such as the ARM Cortex-A720 [21], which are widely used in mobile and edge computing environments with high computational demands. Therefore, optimizing HQC on the ARMv9-A architecture is the primary motivation of this work.

B. Contributions

While existing works primarily focus on polynomial multiplication optimization, the encoding, decoding and fixed-weight sampling operations have received relatively little attention. In this paper, we present the first optimized implementation of HQC on the ARMv9-A architecture, optimizing polynomial multiplication as well as these operations. The main contributions are summarized as follows:

- 111
- 222
- 333

The rest of this paper is organized as follows. Section II introduces the necessary preliminaries. Section III presents the proposed optimization methods of HQC on the ARMv9-A architecture. Section IV provides the performance evaluation and analysis. Section V concludes the paper.

II. Preliminaries

A. Hamming Quasi-Cyclic (HQC)

HQC is a key encapsulation mechanism (KEM) derived from the public key encryption scheme HQC-PKE via the Fujisaki-Okamoto transformation [22], adhering to the HHK framework [23]. It is an IND-CCA2 secure scheme whose security relies on the hardness of the Quasi-Cyclic Syndrome Decoding (QCSD) problem. HQC provides three parameter sets, namely HQC-1, HQC-3 and HQC-5, which correspond to the NIST security levels 1, 3 and 5, respectively, as detailed in Table I. The polynomial arithmetic in HQC is performed over the ring $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$, where n determines the dimension of this ring. The parameters w, w_r and w_e denote the fixed Hamming weights for the sparse polynomial selections. We omit the detailed parameters of the concatenated Reed-Muller and Reed-Solomon codes, which can be referred to in [9]. The comprehensive algorithmic description and implementation details of the HQC-PKE scheme, as well as the derived HQC-KEM scheme, are also available in [9]. For clarity, we outline only the core computational steps in Algorithm 1. The encapsulation key $\mathrm{ek}_{\mathrm{KEM}}$ is identical to $\mathrm{ek}_{\mathrm{PKE}}$, while the decapsulation key $\mathrm{dk}_{\mathrm{KEM}}$ is composed of $(\mathrm{ek}_{\mathrm{KEM}}, \mathrm{dk}_{\mathrm{PKE}}, \sigma, \mathrm{seed}_{\mathrm{KEM}})$, where σ is a randomly generated value, and $\mathrm{seed}_{\mathrm{KEM}}$ is a 32-byte seed. The resulting shared secret key has a fixed size of 32 bytes. While a 32-byte compressed varient of $\mathrm{dk}_{\mathrm{KEM}}$ is also available, we default to the uncompressed form in this paper.

Algorithm 1 Core steps of HQC-PKE

HQC-PKE.Keygen($\mathrm{seed}_{\mathrm{PKE}}$):
$(\mathrm{seed}_{\mathrm{PKE.dk}}, \mathrm{seed}_{\mathrm{PKE.ek}}) \leftarrow \mathsf{I}(\mathrm{seed}_{\mathrm{PKE}})$
$y, x \leftarrow \mathsf{SampleFixedWeightVect}_{\$}(\mathrm{seed}_{\mathrm{PKE.dk}}, \mathcal{R}_w)$
$h \leftarrow \mathsf{SampleVect}(\mathrm{seed}_{\mathrm{PKE.ek}}, \mathcal{R})$
$s \leftarrow x + h \cdot y$
return $(\mathrm{ek}_{\mathrm{PKE}} = (\mathrm{seed}_{\mathrm{PKE.ek}}, s),\ \mathrm{dk}_{\mathrm{PKE}} = \mathrm{seed}_{\mathrm{PKE.dk}})$
HQC-PKE.Encrypt($\mathrm{ek}_{\mathrm{PKE}}, m, \theta$):
$h \leftarrow \mathsf{SampleVect}(\mathrm{seed}_{\mathrm{PKE.ek}}, \mathcal{R})$
$r_1, r_2 \leftarrow \mathsf{SampleFixedWeightVect}(\theta, \mathcal{R}_{w_r})$
$e \leftarrow \mathsf{SampleFixedWeightVect}(\theta, \mathcal{R}_{w_e})$
$u \leftarrow r_1 + h \cdot r_2$
$v \leftarrow \mathcal{C}.\mathsf{Encode}(m) + \mathsf{Truncate}(s \cdot r_2 + e, \ell)$
return $\mathrm{c}_{\mathrm{PKE}} = (u, v)$
HQC-PKE.Decrypt($\mathrm{dk}_{\mathrm{PKE}}, \mathrm{c}_{\mathrm{PKE}}$):
$y \leftarrow \mathsf{SampleFixedWeightVect}_{\$}(\mathrm{seed}_{\mathrm{PKE.dk}}, \mathcal{R}_w)$
$m \leftarrow \mathcal{C}.\mathsf{Decode}\big(v - \mathsf{Truncate}(u \cdot y, \ell)\big)$
return m

TABLE I: HQC parameter sets: Key and ciphertext sizes in bytes

	n	w	$w_r = w_e$	$\lvert\mathrm{ek}_{\mathrm{KEM}}\rvert$	$\lvert\mathrm{dk}_{\mathrm{KEM}}\rvert$	$\lvert\mathrm{c}_{\mathrm{KEM}}\rvert$
HQC-1	17669	66	75	2241	2321	4433
HQC-3	35851	100	114	4514	4602	8978
HQC-5	57637	131	149	7237	7333	14421

B. Scalable Vector Extension 2 (SVE2)

SVE2 [11] is an extension of the ARMv9-A architecture designed to scale parallel computing performance. Based on the original SVE [24], which primarily targets high-performance computing (HPC), SVE2 extends the instruction set to accelerate data processing in fields such as computer vision, machine learning, signal processing and cryptography. It provides 32 scalable vector registers (`z0`–`z31`) for storing variables and 16 predicate registers (`p0`–`p15`), which are utilized to control individual lanes within the vector registers and support complex conditional loops. A defining feature of SVE2 is its Vector Length Agnostic (VLA) programming paradigm. Unlike traditional fixed-width SIMD instructions such as NEON, VLA allows developers to write vectorized code without assuming a specific register size. The hardware dynamically adapts the execution based on its physical vector length, which ranges from 128 to 2048 bits. This ensures that

the code can run directly across different hardware configurations without requiring any modifications. In this work, we target a 128-bit SVE2 implementation that can operate concurrently with NEON [25]. This differs from the Streaming SVE mode in [12], [13], where the vector length expands to 512 bits, but the NEON instructions are strictly disabled.

Algorithm 2 The `Btfy` Algorithm

Require: Polynomial coefficient array $f = [f_0, f_1, \ldots, f_{N-1}]$ of length $N = 2^l$ in the novel polynomial basis, field offset $\alpha \in \mathbb{F}_{2^m}$
Ensure: Evaluation values of f at 2^l points in $\alpha + V_l$
1: **if** $N = 1$ **then**
2: **return** $[f_0]$
3: **end if**
4: $K \leftarrow N/2$ {Half-length of the current recursive segment}
5: $k \leftarrow \log_2(K)$
6: $\beta \leftarrow s_k(\alpha)$ {Precompute the vanishing polynomial scalar}
7: **for** $j = 0$ **to** $K - 1$ **do**
8: $f_j \leftarrow f_j + \beta \cdot f_{j+K}$
9: $f_{j+K} \leftarrow f_{j+K} + f_j$
10: **end for**
11: $E_L \leftarrow \texttt{Btfy}([f_0, \ldots, f_{K-1}], \alpha)$
12: $E_H \leftarrow \texttt{Btfy}([f_K, \ldots, f_{N-1}], \alpha + v_k)$
13: **return** (E_L, E_H)

C. Additive Fast Fourier Transform (AFFT)

In recent work [16], [20], [26], AFFT has been used to accelerate polynomial multiplication in HQC. It efficiently evaluates and interpolates polynomials over finite fields of characteristic two ($\mathbb{F}_{2^m}$). Instead of utilizing roots of unity, it operates over an affine subspace constructed from a Cantor basis [27]. In this work, we focus on the finite extension field $\mathbb{F}_{2^{64}} := \mathbb{F}_2[x]/(x^{64} + x^4 + x^3 + x + 1)$.

Definition 1(Cantor Basis and Vanishing Polynomial). A Cantor basis for $\mathbb{F}_{2^m}$ is defined as an ordered set of m field elements $\{v_0, v_1, \ldots, v_{m-1}\}$ that spans $\mathbb{F}_{2^m}$ as a vector space over $\mathbb{F}_2$, satisfying the relation:

$$v_0 = 1, \ v_{i+1}^2 + v_{i+1} = v_i, \ \text{for } 0 \le i < m - 1.$$

Let $V_i = \text{span}\{v_0, \ldots, v_{i-1}\}$ be a linear subspace of size 2^i. The vanishing polynomial $s_i(x)$ over V_i is defined as:

$$s_i(x) = \prod_{j \in V_i} (x - j).$$

Owing to the algebraic properties of the Cantor basis, $s_i(x)$ possesses a sparse linearized structure satisfying $s_i(x) = s_{i-1}(s_1(x))$, which reduces to the form $s_i(x) = x^{2^i} + x$ whenever i is a power of two. This sparsity eliminates the need for general polynomial divisions, allowing the modular reduction steps to be executed via a minimal number of shifts and XOR operations.

Definition 2(Novel Polynomial Basis). To accelerate the computation of AFFT, a novel polynomial basis [28] is adopted. For an index i expressed in its binary representation $i = \sum_{j \ge 0} i_j \cdot 2^j$, where $i_j \in \{0, 1\}$, the basis polynomial $X_i(x)$ of degree i is defined as:

$$X_i(x) = \prod_{j \ge 0} s_j(x)^{i_j}.$$

By converting the standard monomial basis to this novel basis, the evaluation over 2^l points of the form $\alpha + \omega$, where $\omega \in V_l$ and $\alpha \in \mathbb{F}_{2^m}$, is efficiently executed via the `Btfy` Algorithm. In summary, the AFFT consists of two steps, namely `BasisCvt` for basis conversion with a complexity of $O(n \log n \log\log n)$ [29] and `Btfy` for evaluation with a complexity of $O(n \log n)$. We omit the process of `BasisCvt` since it is not the primary optimization focus of this paper, and we refer readers to [20], [29], [30] for further details.

III. Methodology

IV. Performance Evaluation

V. Conclusion

In this paper, we present the first optimized implementation of HQC on the ARMv9-A architecture. Experimental results show that our implementation outperforms the state-of-the-art NEON implementation, demonstrating the effectiveness of our proposed optimizations. This work provides a solid performance baseline and lays a foundation for future PQC optimizations on emerging ARM platforms. Future work will focus on further optimizing polynomial multiplication and other operations, as well as exploring optimized implementations of other PQC algorithms on the ARMv9-A architecture.

References

[1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.
[2] G. Alagic, Q. Dang, D. Moody, A. Robinson, H. Silberg, D. Smith-Tone, *et al.*, "Module-lattice-based key-encapsulation mechanism standard," 2024.
[3] T. Dang, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, *et al.*, "Module-lattice-based digital signature standard," 2024.
[4] D. Cooper *et al.*, "Stateless hash-based digital signature standard," 2024.
[5] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, Z. Zhang, *et al.*, "Fast-fourier lattice-based compact signatures over ntru," *URL: https://www. di. ens. fr/˜ prest/Publications/falcon. pdf (: 5.04. 2024)*, 2019.
[6] G. Alagic, M. Bros, P. Ciadoux, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, *et al.*, *Status report on the fourth round of the nist post-quantum cryptography standardization process*. US Department of Commerce, National Institute of Standards and Technology ..., 2025.
[7] M. R. Albrecht, D. J. Bernstein, T. Chou, C. Cid, J. Gilcher, T. Lange, V. Maram, I. Von Maurich, R. Misoczki, R. Niederhagen, *et al.*, "Classic mceliece: conservative code-based cryptography," 2022.
[8] N. Aragon, P. Barreto, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, S. Ghosh, S. Gueron, T. Güneysu, *et al.*, "Bike: bit flipping key encapsulation," 2022.
[9] P. Gaborit, C. Aguilar-Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, E. Persichetti, G. Zémor, J. Bos, *et al.*, "Hamming quasi-cyclic (hqc)," *NIST Post-Quantum Standardization, 4th Round; National Institute of Standards and Technology: Gaithersburg, MD, USA*, 2025.
[10] ARM Ltd., "Arm architecture reference manual supplement, Armv9-A." https://developer.arm.com/documentation/ddi0608/latest/, 2021.
[11] ARM Ltd., "Learn the architecture - introducing SVE2 guide." https://developer.arm.com/documentation/102340/0100/?lang=en, 2024.

[12] H. Wei, W. Li, S. Shen, H. Yang, and Y. Zhao, "Optimized implementation of ml-kem on armv9-a with sve2 and sme," *Cryptology ePrint Archive*, 2026.
[13] H. Wei, W. Li, S. Shen, H. Yang, W. Guo, and Y. Zhao, "Vectorized sve2 optimization of the post-quantum signature ml-dsa on armv9-a architecture," *Cryptology ePrint Archive*, 2025.
[14] J.-M. Robert and P. Véron, "Faster multiplication over f 2 [x] using avx512 instruction set and vpclmulqdq instruction," *Journal of Cryptographic Engineering*, vol. 13, no. 1, pp. 37–55, 2023.
[15] B. Dong, H. Feng, and Q. Wang, "Opthqc: Optimize hqc for high-performance post-quantum cryptography," *arXiv preprint arXiv:2512.12904*, 2025.
[16] M.-S. Chen, C.-M. Chiu, C.-T. Peng, and B.-Y. Yang, "Accelerating hqc with additive fft," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2026, no. 2, pp. 520–544, 2026.
[17] M.-S. Chen, T. Chou, and M. Krausz, "Optimizing bike for the intel haswell and arm cortex-m4," *Cryptology ePrint Archive*, 2021.
[18] M. Lee, J. Jang, S. Kim, and S. Hong, "Improved frobenius fft for code-based cryptography on cortex-m4," *IEEE Internet of Things Journal*, 2025.
[19] D. Kim, J. Choi, S. Yoon, and S. C. Seo, "Optimized implementation of hqc on cortex-m4," *ICT Express*, 2025.
[20] J. Jang, M. Lee, D. Kwon, S. Hong, S. Kim, and S. Lee, "Efficient polynomial multiplication for hqc on arm cortex-m4," *Cryptology ePrint Archive*, 2025.
[21] ARM Ltd., "Arm Cortex-A720 processor." https://developer.arm.com/Processors/Cortex-A720, 2023.
[22] E. Fujisaki and T. Okamoto, "Secure integration of asymmetric and symmetric encryption schemes," in *Annual international cryptology conference*, pp. 537–554, Springer, 1999.
[23] D. Hofheinz, K. Hövelmanns, and E. Kiltz, "A modular analysis of the fujisaki-okamoto transformation," in *Theory of Cryptography Conference*, pp. 341–371, Springer, 2017.
[24] N. Stephens, S. Biles, M. Boettcher, J. Eapen, M. Eyole, G. Gabrielli, M. Horsnell, G. Magklis, A. Martinez, N. Premillieu, *et al.*, "The arm scalable vector extension," *IEEE micro*, vol. 37, no. 2, pp. 26–39, 2017.
[25] ARM Ltd., "Learn the architecture - migrate NEON to SVE." https://developer.arm.com/documentation/102131/0100/?lang=en, 2022.
[26] M.-S. Chen, T.-Y. Chien, C.-M. Chiu, H.-H. Lin, C.-T. Peng, and B.-Y. Yang, "Additive ffts for hqc on arm cortex-m4, revisited," *Cryptology ePrint Archive*, 2026.
[27] D. G. Cantor, "On arithmetical algorithms over finite fields," *Journal of Combinatorial Theory, Series A*, vol. 50, no. 2, pp. 285–300, 1989.
[28] S.-J. Lin, W.-H. Chung, and Y. S. Han, "Novel polynomial basis and its application to reed-solomon erasure codes," in *2014 ieee 55th annual symposium on foundations of computer science*, pp. 316–325, IEEE, 2014.
[29] S.-J. Lin, T. Y. Al-Naffouri, Y. S. Han, and W.-H. Chung, "Novel polynomial basis with fast fourier transform and its application to reed–solomon erasure codes," *IEEE Transactions on Information Theory*, vol. 62, no. 11, pp. 6284–6299, 2016.
[30] S.-J. Lin, T. Y. Al-Naffouri, and Y. S. Han, "Fft algorithm for binary extension finite fields and its application to reed–solomon codes," *IEEE Transactions on Information Theory*, vol. 62, no. 10, pp. 5343–5358, 2016.